Reproducible Data Analysis

James Leveque

2026-04-16

1 Busiest Airports Analysis

1.1 Overview

Global air travel tells a compelling story of disruption and recovery. This section examines annual passenger traffic across six of the world's busiest airports — Atlanta (ATL), Frankfurt (FRA), Beijing Daxing (PKX), Dubai (DXB), London Heathrow (LHR), and Los Angeles (LAX) — from 2020 through 2025. The data were sourced from Wikipedia and the Airports Council International (ACI).

1.2 Passenger Traffic Table

Table 1: Busiest Passenger Airports (2020–2025): Total passengers in millions. Source: Wikipedia / ACI.

Airport	2020	2021	2022	2023	2024	2025
Atlanta (ATL)	42.92	75.70	93.70	104.65	108.07	106.30
Beijing Daxing (PKX)	NA	25.05	NA	49.44	49.44	53.62
Dubai (DXB)	25.84	29.11	66.07	86.99	92.33	95.20
Frankfurt (FRA)	18.77	24.81	48.92	59.36	61.56	63.19
London Heathrow (LHR)	22.11	NA	61.61	79.18	83.88	84.46
Los Angeles (LAX)	28.78	48.01	65.92	75.05	76.59	73.71

Table 1 presents total annual passengers (in millions) for each airport across the six-year window. Several patterns stand out immediately. Atlanta consistently leads the pack, handling over 100 million passengers annually by 2023 — a scale that dwarfs every other airport in the dataset. Dubai's trajectory is equally striking: starting from a pandemic low of just 25.8 million in 2020, it climbed to 95.2 million by 2025, making it the second-busiest airport in the dataset. Notably, Beijing Daxing and London Heathrow each have missing values in certain years, reflecting either data unavailability or the airport falling outside the Wikipedia top-50 ranking in those years.

1.3 Passenger Traffic Over Time

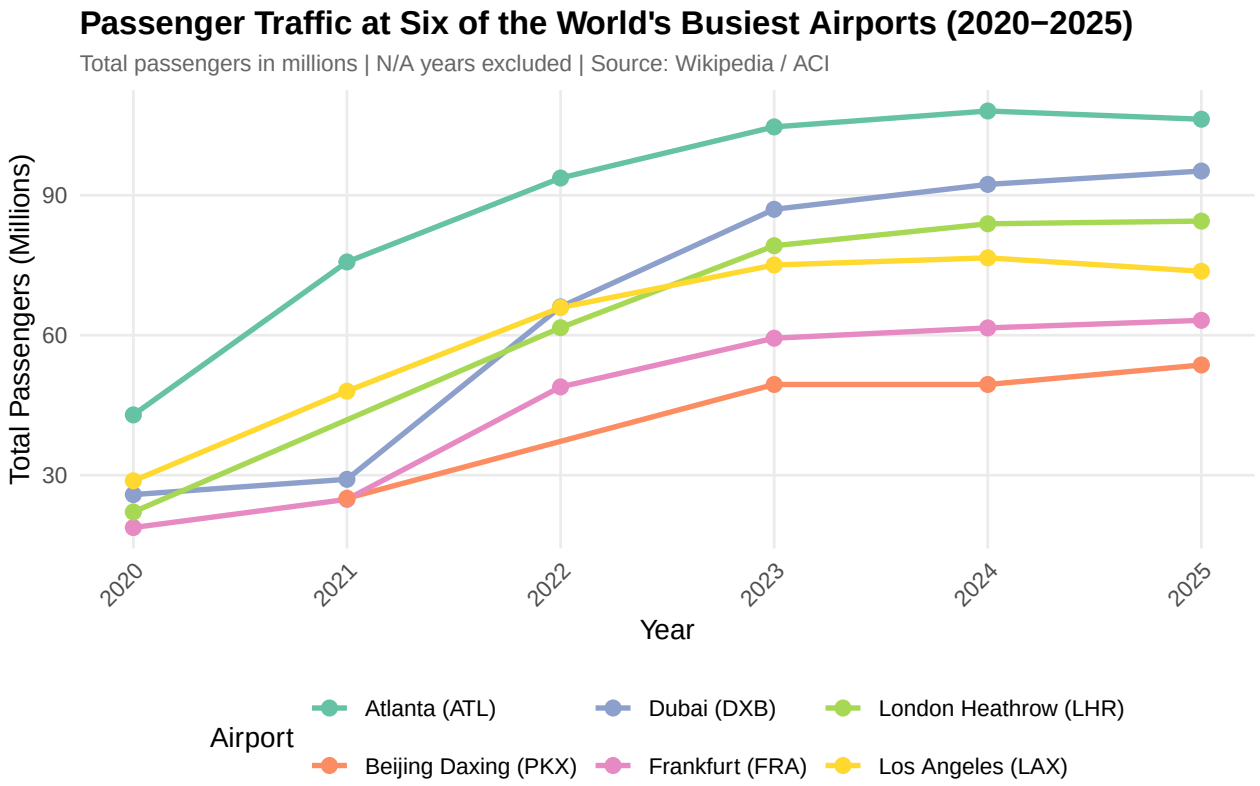


Figure 1: Passenger traffic at six of the world's busiest airports from 2020 to 2025. Years with no available data are excluded. Source: Wikipedia / ACI.

Figure 1 makes one thing immediately apparent: the COVID-19 pandemic devastated global air travel in 2020, and the recovery since then has been dramatic but uneven across airports. Every airport in the dataset hit a historic low in 2020, but the speed and shape of recovery differed significantly depending on the airport's location and the travel policies of its home country.

Atlanta's dominance throughout the chart tells an important story. Even at its 2020 pandemic low of 42.9 million passengers, Atlanta still handled more traffic than Frankfurt or Dubai did in most years. Its strong domestic travel base insulated it from the worst of the international travel collapse, and it recovered faster than any other airport in the dataset. Dubai, by contrast, is almost entirely dependent on international travel, yet its steep recovery curve — driven by aggressive tourism promotion and its position as a global hub — is arguably more impressive in relative terms. Placing these two airports alongside each other in the same chart makes their very different recovery strategies visible in a way that a table of numbers alone cannot convey.

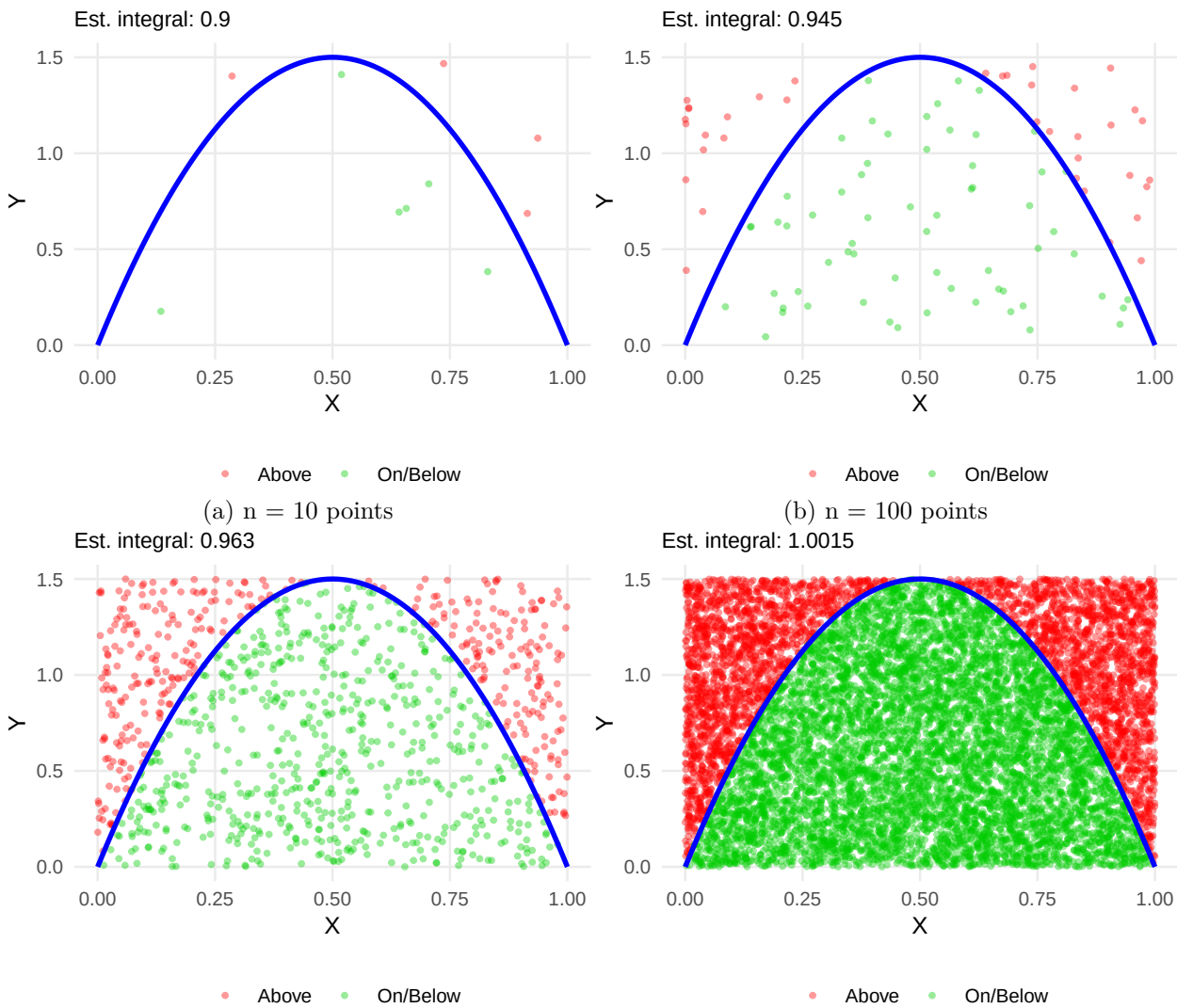
2 Monte Carlo Numerical Integration

2.1 The Monte Carlo Method

Monte Carlo numerical integration is a technique for approximating the value of a definite integral by using random sampling. Rather than evaluating the function analytically, the method works by randomly scattering points within a rectangular bounding window and counting what fraction land on or below the curve. Multiplying that fraction by the area of the bounding rectangle yields an estimate of the integral. The key insight is that as the number of randomly sampled points increases, the estimate converges toward the true value — a direct consequence of the Law of Large Numbers.

For this analysis, the function being integrated is the probability density function of the Beta distribution with shape parameters $\alpha = 2$ and $\beta = 2$, given by $f(x) = \text{dbeta}(x, 2, 2)$, over the interval $x \in [0, 1]$. The bounding rectangle spans $x \in [0, 1]$ and $y \in [0, 1.5]$, giving a window area of $1 \times 1.5 = 1.5$.

2.2 Small Multiple Visualization



(a) n = 10 points (b) n = 100 points

(c) n = 1,000 points (d) n = 10,000 points

Figure 2: Monte Carlo numerical integration of the Beta(2,2) PDF at four resolutions.

2.3 Discussion

Figure 2 reveals how the accuracy and stability of the Monte Carlo estimate change dramatically with resolution. At $n = 10$, the estimate is highly unreliable — with only ten points scattered across the bounding rectangle, the proportion landing on or below the Beta curve is essentially a coin flip, and the estimate could differ substantially each time the simulation is run. By $n = 100$, the point cloud begins to take shape, though noticeable variability remains. At $n = 1{,}000$ and especially $n = 10{,}000$, the green points trace the bell-shaped profile of the Beta(2,2) PDF with remarkable fidelity, and the estimated integral stabilizes consistently.

The exact value of $\int_0^1 \text{dbeta}(x, 2, 2)\, dx$ is **1.0**, since the integral of any valid probability density function over its entire support must equal 1 by definition. The Monte Carlo estimates at $n = 10{,}000$

converge very close to this value, providing visual confirmation of both the method's correctness and its dependence on sufficient sample size. This exercise illustrates a broader principle: the Monte Carlo method is not just a computational trick, but a demonstration of how randomness, when applied at scale, can yield highly accurate numerical results.

3 Planning and Prompting GenAI Tools

3.1 Data Context and Plan

This section explores how the quality of a prompt given to a generative AI tool affects the quality of the output it produces. The dataset used here is `calcium.csv`, which contains measurements of ulnar calcium levels from 31 women tracked over four years. Fifteen women made up a null treatment group (no intervention), while 16 women participated in a diet and exercise program. Measurements were taken at baseline and at the end of Years 1, 2, and 3.

The data as provided are not tidy: the two treatment groups occupy columns side by side, with duplicate column names (`y0, y1, y2, y3, y0, y1, y2, y3`). Before any visualization can be created, the data need to be reshaped into a long format where each row represents a single observation for one woman at one time point.

Plan developed for gen AI prompting:

Goal: Tidy the calcium dataset and create a visualization that allows comparison of ulnar calcium levels across time and between the two treatment groups (Null and Diet/Exercise).

Needs: R with `tidyverse`; the `calcium.csv` file.

Steps:

1. Load the CSV while skipping the original header row and assigning correct column names manually, since the file contains duplicate names.
2. Separate the two groups into distinct data frames, adding a `group` label ("Null" or "Diet/Exercise") and a subject ID to each.
3. Pivot each group from wide to long format so that each row captures one subject at one time point, with columns for `subject`, `group`, `year`, and `calcium`.
4. Bind the two long data frames into a single tidy data frame.
5. Recode year labels from `y0`–`y3` to "Initial," "Year 1," "Year 2," and "Year 3" and treat them as an ordered factor.
6. Create a line plot showing individual women's trajectories over time, faceted by group, with a bold group mean line overlaid to make the overall trend legible alongside individual variation.

3.2 What response Claude gave:

The following R code was produced by Anthropic Claude (Sonnet 4.6) after being given the plan above along with the `calcium.csv` file as an attachment.

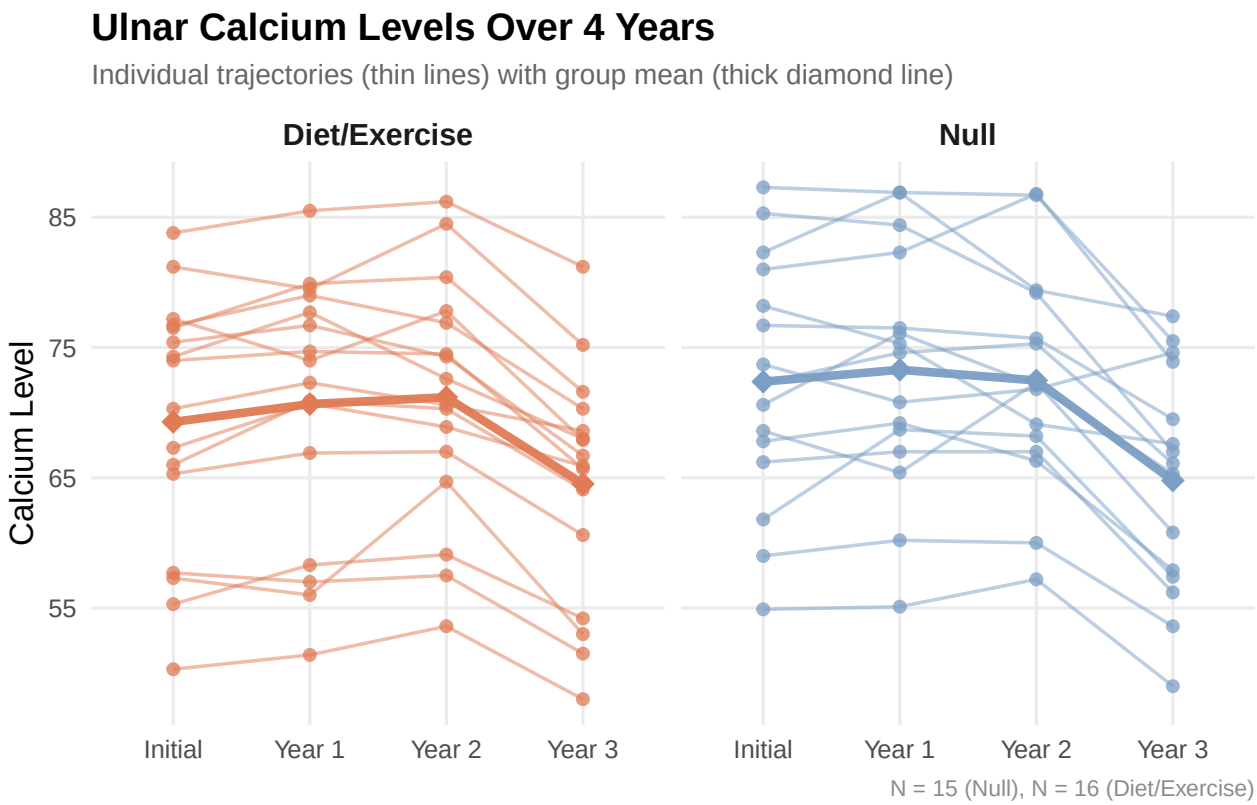


Figure 3: Ulnar calcium levels over four years by treatment group. Individual trajectories (thin lines) are shown alongside group means (thick diamond line). Plan-informed Claude response.

3.3 Generic Claude Response

The following output was produced by Anthropic Claude (Sonnet 4.6 Extended) using a generic prompt: *"Create a data visualization that will help me understand these data."* The CSV file was attached without any additional context or plan.

The tool's written response was as follows: both groups follow a similar pattern — values remain relatively stable across the first three time points before dropping noticeably at the final measurement. Group A starts slightly higher at baseline (mean ~72.4 vs. ~69.3 for Group B), but both groups converge to nearly identical final values (~64.8 and ~64.5). The tool also noted substantial individual variation, with some participants staying high throughout while others showed unusual trajectories. Importantly, the generic response referred to the groups only as "Group A" and "Group B," as no context about the nature of the treatment was provided in the prompt.

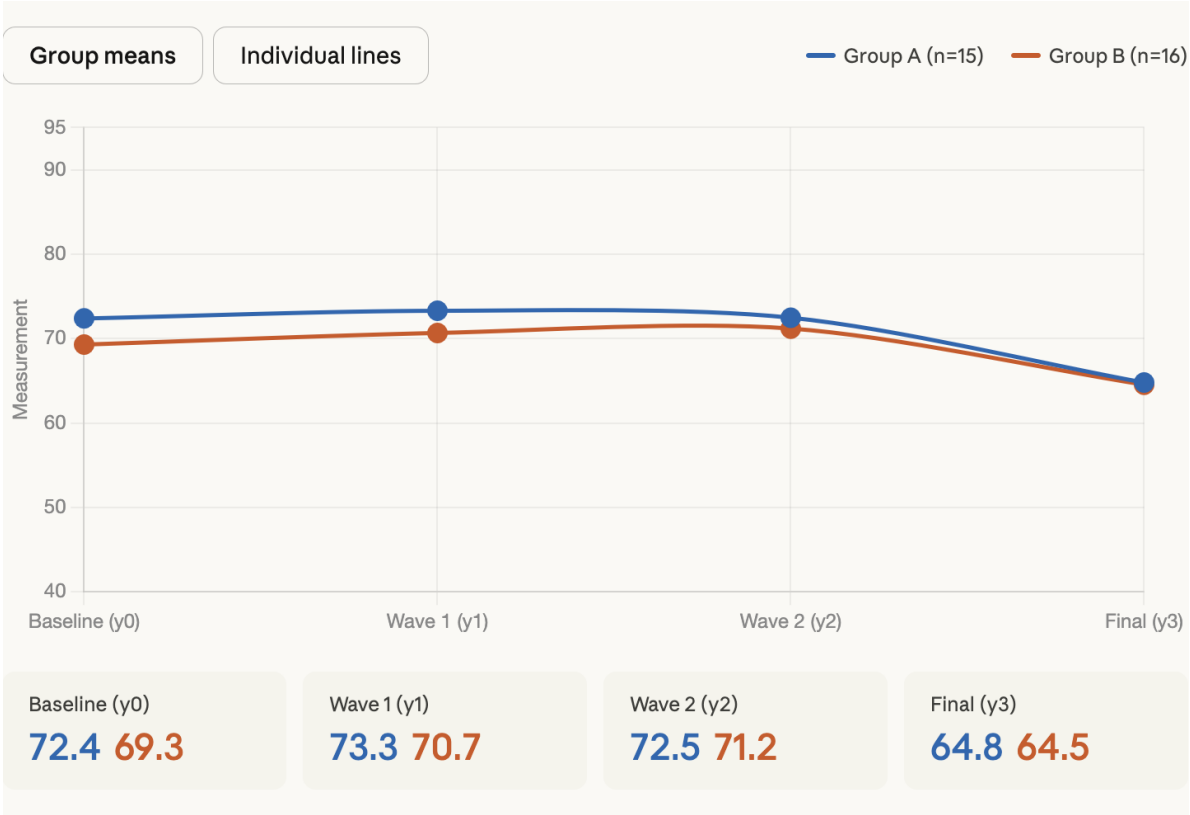


Figure 4: Generic Claude response — Image 1: Individual trajectories and group means for Group A (n=15) and Group B (n=16). Labels use y0–y3 rather than descriptive time labels.

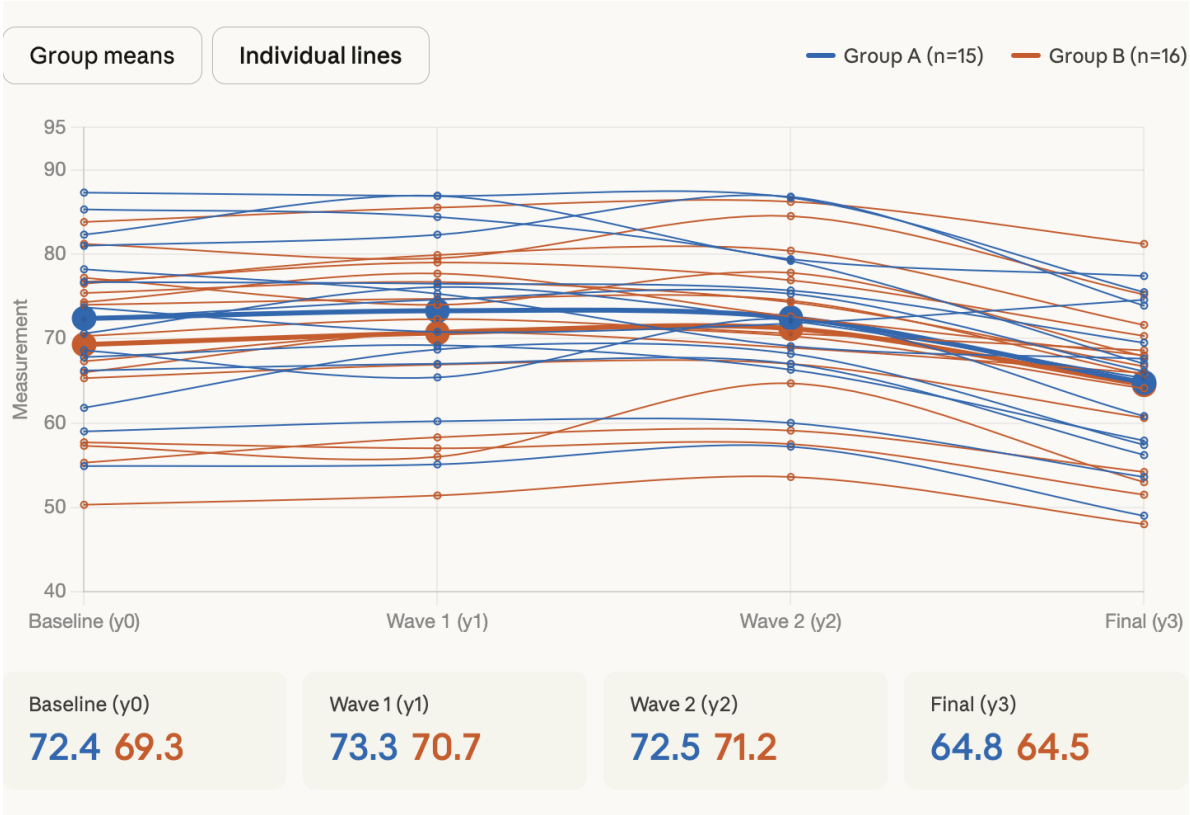


Figure 5: Generic Claude response — Image 2: Group means only for Group A (n=15) and Group B (n=16), showing the overall declining trend.

3.4 Comparing the Two Approaches

The contrast between these two sets of results is instructive. The plan-informed response produced a plot (Figure 3) that is immediately interpretable: the groups are labeled with their actual names ("Null" and "Diet/Exercise"), the time axis uses human-readable labels ("Initial" through "Year 3"), and the faceted layout with an overlaid group mean makes it easy to compare both individual variation and aggregate trends simultaneously. The tidy data structure was also handled correctly because the plan explicitly described the problem — duplicate column headers and side-by-side groups — and prescribed exact steps for resolving it.

The generic response (Figure 4 and Figure 5), while visually polished and technically correct, tells a fundamentally less informative story. The groups are labeled "Group A" and "Group B" — the tool had no basis for knowing which was the treatment group — and the time axis retains the raw variable names (`y0`, `y1`, `y2`, `y3`) rather than meaningful labels. The tool also produced two separate plots (one with individual lines, one with means only) rather than a single integrated view. Crucially, the generic response correctly identified that both groups decline sharply at the final time point and that individual variation is substantial — observations that are accurate but surface-level, because the tool had no context about the study design to reason from.

The broader lesson here is that generative AI tools are sensitive to the specificity and context embedded in a prompt. A well-constructed plan does not just yield better-labeled axes; it shapes

what the tool understands the problem to be, which in turn shapes every decision the tool makes about how to structure, wrangle, and display the data.

4 Reflection

4.1 What I Have Learned

This course has pushed me to think more carefully about what it means to work with data rigorously and reproducibly, and several specific skills stand out as genuinely transformative.

The most foundational shift has been learning to think about tidy data. Before this course, I would have looked at the `calcium.csv` file — with its side-by-side columns and duplicate headers — and felt uncertain about how to proceed. Now I can recognize immediately that the data violates the principle of one observation per row and one variable per column, and I have a concrete toolkit for fixing it: separating groups, pivoting to long format, binding, and recoding. The wrangling code I wrote for the calcium dataset in this assignment is a direct demonstration of that growth.

Data visualization has also become something I approach with much more intentionality. Early in the course, I thought of a plot primarily as a way to display numbers. Now I think about what story a visualization should tell, what the reader's eye will be drawn to first, and how design choices — color, geometry, faceting, axis labels — either support or undermine that story. The airports line plot is a good example: the decision to overlay points on lines, exclude missing years rather than interpolate, and use a colorblind-friendly palette were all deliberate choices aimed at making the data more honest and more readable.

Finally, learning Quarto has changed how I think about the relationship between analysis and communication. The ability to write narrative text alongside executable code — and have both rendered into a single, professional document — makes it much harder to treat analysis and reporting as separate activities. When the code and the words live in the same file, it becomes natural to ask whether the narrative actually reflects what the code is doing, and to revise both together. That habit of integrated thinking feels like one of the most durable things I will take from this course.

Appendix A: GenAI Usage

Tool Information

Tool: Anthropic Claude **Version:** Claude Sonnet 4.6 Extended **Date of Use:** March 31, 2026

Plan-Informed Prompt

The following prompt was submitted to Anthropic Claude along with the `calcium.csv` file as an attachment:

> I have a dataset about ulnar calcium levels measured in 31 women over 4 years. 15 women were in a null treatment group and 16 were in a diet/exercise treatment group. The CSV has 8 columns — the first four (null_y0, null_y1, null_y2, null_y3) belong to the null group and the last four (diet_y0, diet_y1, diet_y2, diet_y3) belong to the diet/exercise group. The data is not tidy.
>
> Here is my plan:
>
> - Separate the two groups, add a group label and subject ID to each
> - Pivot each group longer so each row is one woman at one time point with columns: subject, group, year, calcium
> - Bind the two long data frames together
> - Convert year labels (y0–y3) to Initial, Year 1, Year 2, Year 3 as an ordered factor
> - Create a line plot showing calcium over time, with individual women as lines, colored or faceted by group
>
> Please write R code using tidyverse to tidy this data and produce this plot.

Plan-Informed Response

The exact code returned by Claude in response to the plan-informed prompt is reproduced in the Code Appendix (see **?@sec-code**).

Generic Prompt

The generic prompt used by the course instructor was:

> Create a data visualization that will help me understand these data. [calcium.csv attached]

Generic Response

The full text response generated by Anthropic Claude (Sonnet 4.6 Extended) using the generic prompt was as follows:

> Here's what stands out in the data:
>
> Both groups follow the same pattern — values stay relatively stable across the first three waves (y0→y1→y2), then drop noticeably at the final measurement (y3). This sharp decline at y3 is the dominant signal.
>
> Group A starts slightly higher (baseline mean ~72.4 vs. ~69.3 for Group B), but both groups converge to nearly identical final values (~64.8 and ~64.5). That convergence is interesting — Group A drops about 7.5 points while Group B drops about 4.8.
>
> There's substantial individual variation — hit the "Individual lines" toggle to see this. Some participants stay high throughout, others are consistently low, and a few show unusual trajectories (one in Group A actually rises at y3 while most fall).
>
> If you can share what the groups represent (e.g., treatment vs. control, or different diets/supplements) and what's being measured, I can dig into the statistics more precisely — things like whether the group difference at y3 is significant, or which individuals are outliers.

The two accompanying images are shown as Figure 4 and Figure 5 in the main body of this document.

Appendix B: Code

```r
#   Libraries
library(tidyverse)
library(knitr)

# ==============================================================================
# SECTION 1: BUSIEST AIRPORTS ANALYSIS
# ==============================================================================

# Data sourced from Wikipedia / ACI
# https://en.wikipedia.org/wiki/List_of_busiest_airports_by_passenger_traffic

airport_data <- tibble(
  airport = rep(c("ATL", "FRA", "PKX", "DXB", "LHR", "LAX"), each = 6),
  name = rep(c(
    "Atlanta (ATL)",
    "Frankfurt (FRA)",
    "Beijing Daxing (PKX)",
    "Dubai (DXB)",
    "London Heathrow (LHR)",
    "Los Angeles (LAX)"
  ), each = 6),
  year = rep(2020:2025, times = 6),
  passengers = c(
    # ATL - Hartsfield-Jackson Atlanta
    42918685, 75704760, 93699630, 104653451, 108067766, 106302208,
    # FRA - Frankfurt Airport
    18768601, 24812849, 48918482, 59355389, 61564957, 63189666,
    # PKX - Beijing Daxing International (NA = not in top 50 that year)
    NA, 25051012, NA, 49441029, 49441029, 53618949,
    # DXB - Dubai International
    25836771, 29110609, 66069981, 86994365, 92331506, 95200000,
    # LHR - London Heathrow
    22111469, NA, 61614508, 79183364, 83884572, 84463000,
    # LAX - Los Angeles International
    28779527, 48007284, 65924298, 75050875, 76588028, 73709594
  )
)

#   Table: passengers in millions, wide format
airport_data |>
  mutate(passengers_millions = round(passengers / 1000000, 2)) |>
  select(name, year, passengers_millions) |>
  pivot_wider(
    names_from  = year,
```

```r
    values_from = passengers_millions
  ) |>
  arrange(name) |>
  kable(
    col.names = c("Airport", "2020", "2021", "2022", "2023", "2024", "2025"),
    align     = c("l", "c", "c", "c", "c", "c", "c")
  )

#   Line plot: traffic trends over time
airport_data |>
  filter(!is.na(passengers)) |>                      # exclude missing years
  mutate(passengers_millions = passengers / 1000000) |>
  ggplot(aes(x = year, y = passengers_millions, color = name, group = name)) +
  geom_line(linewidth = 1) +
  geom_point(size = 2.5) +
  scale_x_continuous(breaks = 2020:2025) +
  scale_y_continuous(labels = scales::comma) +
  scale_color_brewer(palette = "Set2") +
  labs(
    title    = "Passenger Traffic at Six of the World's Busiest Airports (2020-2025)",
    subtitle = "Total passengers in millions | N/A years excluded | Source: Wikipedia / ACI",
    x        = "Year",
    y        = "Total Passengers (Millions)",
    color    = "Airport"
  ) +
  theme_minimal() +
  theme(
    plot.title       = element_text(face = "bold", size = 13),
    plot.subtitle    = element_text(size = 9, color = "gray40"),
    legend.position  = "bottom",
    panel.grid.minor = element_blank(),
    axis.text.x      = element_text(angle = 45, hjust = 1)
  )

# ============================================================================
# SECTION 2: MONTE CARLO NUMERICAL INTEGRATION
# ============================================================================

# Function: dbeta(x, shape1 = 2, shape2 = 2)
# x bounds: [0, 1]  |  y bounds: [0, 1.5]
# Bounding rectangle area: 1 * 1.5 = 1.5
# True integral value: 1.0 (sum of a valid PDF over its full support)

#   Simulation function
# Generates n random (x, y) pairs within the bounding rectangle
monte_carlo_sim <- function(n, x_min = 0, x_max = 1,
                            y_min = 0, y_max = 1.5) {
```

```r
  tibble(
    x = runif(n = n, min = x_min, max = x_max),
    y = runif(n = n, min = y_min, max = y_max)
  )
}

#   Single-resolution plot function
# Wraps wrangling + ggplot for one value of n
mc_plot <- function(n_points) {
  sim_data <- monte_carlo_sim(n = n_points) |>
    mutate(
      f_x  = dbeta(x, shape1 = 2, shape2 = 2),   # evaluate PDF at each x
      flag = if_else(y <= f_x, "On/Below", "Above") # classify each point
    )

  # Estimate integral: proportion on/below * window area
  window_area  <- 1 * 1.5
  p            <- mean(sim_data$flag == "On/Below")
  est_integral <- round(p * window_area, 4)

  ggplot(sim_data, aes(x = x, y = y, color = flag)) +
    geom_point(alpha = 0.4, size = 0.8) +
    stat_function(                                  # overlay true PDF curve
      fun       = dbeta,
      args      = list(shape1 = 2, shape2 = 2),
      xlim      = c(0, 1),
      color     = "blue",
      linewidth = 1
    ) +
    scale_color_manual(values = c("Above" = "red", "On/Below" = "green3")) +
    labs(
      subtitle = paste0("Est. integral: ", est_integral),
      x = "X", y = "Y", color = NULL
    ) +
    theme_minimal(base_size = 10) +
    theme(
      plot.subtitle    = element_text(size = 9),
      legend.position  = "bottom",
      panel.grid.minor = element_blank()
    )
}

# Render all four panels (set seed for reproducibility)
set.seed(42)
mc_plot(10)
mc_plot(100)
mc_plot(1000)
```

```r
mc_plot(10000)

# ==============================================================================
# SECTION 3: CALCIUM DATA - PLAN-INFORMED CLAUDE RESPONSE
# ==============================================================================

#   Read raw CSV
# Header row has duplicate names (y0,y1,y2,y3,y0,y1,y2,y3), so i skipped it
# and assigned column names manually
raw <- read_csv(
  "calcium.csv",
  col_names = FALSE,
  skip = 1
)

#   Separate null treatment group (columns 1-4)
null_wide <- raw |>
  select(1:4) |>
  set_names(c("y0", "y1", "y2", "y3")) |>
  filter(!is.na(y0)) |>                          # row 16 is empty for null group
  mutate(subject = paste0("N", row_number()),
         group   = "Null")

#   Separate diet/exercise group (columns 5-8)
diet_wide <- raw |>
  select(5:8) |>
  set_names(c("y0", "y1", "y2", "y3")) |>
  mutate(subject = paste0("D", row_number()),
         group   = "Diet/Exercise")

#   Pivot each group to long format
null_long <- null_wide |>
  pivot_longer(cols = y0:y3, names_to = "year_raw", values_to = "calcium")

diet_long <- diet_wide |>
  pivot_longer(cols = y0:y3, names_to = "year_raw", values_to = "calcium")

#   Bind and recode year labels
year_labels <- c(y0 = "Initial", y1 = "Year 1", y2 = "Year 2", y3 = "Year 3")

calcium_long <- bind_rows(null_long, diet_long) |>
  mutate(
    year = factor(
      year_labels[year_raw],
      levels  = c("Initial", "Year 1", "Year 2", "Year 3"),
      ordered = TRUE
    )
```

```r
  ) |>
  select(subject, group, year, calcium)

#   Line plot with individual trajectories and group mean
group_colors <- c("Null" = "#7B9EC4", "Diet/Exercise" = "#E07B54")

ggplot(calcium_long,
       aes(x = year, y = calcium, group = subject, color = group)) +
  geom_line(alpha = 0.50, linewidth = 0.65) +
  geom_point(size = 1.8, alpha = 0.75) +
  stat_summary(aes(group = group), fun = mean, geom = "line",  # group mean line
               linewidth = 1.6, alpha = 0.95) +
  stat_summary(aes(group = group), fun = mean, geom = "point", # group mean points
               size = 4, shape = 18) +
  facet_wrap(~ group) +
  scale_color_manual(values = group_colors, guide = "none") +
  scale_y_continuous(breaks = seq(45, 95, by = 10)) +
  labs(
    title    = "Ulnar Calcium Levels Over 4 Years",
    subtitle = "Individual trajectories (thin lines) with group mean (thick diamond line)",
    x        = NULL,
    y        = "Calcium Level",
    caption  = "N = 15 (Null), N = 16 (Diet/Exercise)"
  ) +
  theme_minimal(base_size = 13) +
  theme(
    plot.title       = element_text(face = "bold", size = 15),
    plot.subtitle    = element_text(color = "grey40", size = 11),
    strip.text       = element_text(face = "bold", size = 12),
    panel.grid.minor = element_blank(),
    plot.caption     = element_text(color = "grey55", size = 9)
  )
```